

Mastering Error Handling in JavaScript



1. SyntaxError

```
console.log("Vikas"
```

Live reload enabled.

✖ Uncaught SyntaxError: missing) after argument list

SyntaxError – Occurs when there is a mistake in the syntax of the JavaScript code.

2. ReferenceError

```
console.log(age); // ReferenceError
```

```
✖ ▶ Uncaught ReferenceError: age is not defined  
  at customReact.js:34:13
```

ReferenceError – Occurs when trying to access a variable that is not defined.

3. TypeError

```
let obj = null;  
  
console.log(obj.property); // TypeError
```

```
✖ ▶ Uncaught TypeError: Cannot read properties of null (reading 'property')  
    at customReact.js:38:17
```

TypeError – Occurs when an operation is performed on a value of the wrong type.

4. RangeError

```
let arr = new Array(-1)
```

```
✖ ▶ Uncaught RangeError: Invalid array length  
   at customReact.js:40:11
```

RangeError – Occurs when a value is outside the allowable range.

5. URIError

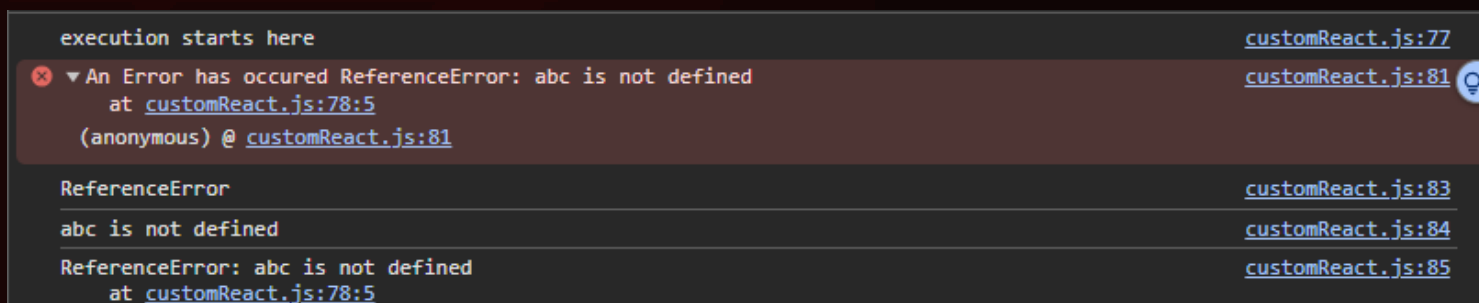
```
decodeURIComponent("%");
```

```
✖ Uncaught URIError: URI malformed  
  at decodeURIComponent (<anonymous>)  
  at customReact.js:75:1
```

[customReact.js:75](#)

Handling Errors With the try **and** catch

```
try {  
  console.log("execution starts here");  
  abc;  
  console.log("execution ends here");  
} catch (err) {  
  console.error("An Error has occurred", err);  
  
  console.log(err.name);    // Name of the error (e.g., ReferenceError)  
  console.log(err.message); // Error message (e.g., "abc is not defined")  
  console.log(err.stack);   // Stack trace (where the error occurred)  
}
```



```
execution starts here                                customReact.js:77  
✖ An Error has occurred ReferenceError: abc is not defined  customReact.js:81  
  at customReact.js:78:5  
  (anonymous) @ customReact.js:81  
  
ReferenceError                                customReact.js:83  
abc is not defined                            customReact.js:84  
ReferenceError: abc is not defined             customReact.js:85  
  at customReact.js:78:5
```

- try is used to wrap code that might throw an error.
- catch captures that error and lets you respond gracefully.
- Without try...catch, the error would crash the script.

Error Handling : Real - World Use

```
function validateAge(age){  
  try{  
    if(isNaN(age)){  
      throw new Error("Invalid input: Age must be number.");  
    }  
    console.log(`user's age is: ${age}`);  
  } catch (err){  
    console.log("validation Error:", err.message);  
  }  
}  
  
validateAge("vikas")
```

✖ validation Error: Invalid input: Age must be number.
validateAge @ customReact.js:97

customReact.js:97



validateAge(age) – What it does:

Checks if age is a number.

If not, throws an error.

Uses try...catch to handle it gracefully.

◆ "vikas" is not a number → throws custom error

◆ Output:

Validation Error: Invalid input: Age must be a number.

Handling JSON

```
function parseJSONSafely(str) {  
  try {  
    return JSON.parse(str);  
  } catch (err) {  
    console.error("Invalid JSON:", err.message);  
    return null;  
  }  
}  
  
const userData = parseJSONSafely('{"name": "tapaScript"}'); // Parsed  
const badData = parseJSONSafely('name: tapaScript'); // Handled gracefully
```

```
✖ Invalid JSON: Unexpected token 'a', "name: javaScript" is not valid JSON customReact.js:111  
parseJSONSafely @ customReact.js:111  
(anonymous) @ customReact.js:117
```

If there's no try...catch, the app will crash after the second line.

API Error Handling

```
async function fetchUserData() {  
  try {  
    const response = await fetch("https://api.example.com/users");  
  
    if (!response.ok) {  
      throw new Error(`HTTP error! Status: ${response.status}`);  
    }  
  
    const data = await response.json();  
    console.log("User data:", data);  
  } catch (err) {  
    console.error("Failed to fetch user data:", err.message);  
    console.log("Error Name:", err.name);  
    console.log("Stack Trace:", err.stack);  
  }  
}
```

- try block wraps your async API call.
- If fetch fails (network issue) or response.ok is false, it throws an error.
 catch handles both:
 Network errors
- Unsuccessful HTTP status codes (like 404 or 500)

Thank you!



Vikas Kushwah